LeadLens Security Practices & Architecture Documentation

LeadLens implements a zero-trust security model. Under this architecture, no endpoint, data query, or system resource serves a single line of data without explicit authentication and authorization validation.

---

1. Core Authentication Model (JWT)

Every request to secure API endpoints must include a valid JSON Web Token (JWT) in the 'Authorization' header as a Bearer token:

'Autho

Token Generation & Lifecycle

- Hashing Context: User passwords are encrypted using 'Bcrypt' with passlib contexts ('CryptContext') before database storage. Raw passwords are never stored or logged.

- Tok
secur

Dependency Injection Enforcement

Every secure route depends on the 'get_current_user' dependency:

1. Dec

---

2. Role-Based Access Control (RBAC)

LeadLens enforces clear segregation of duties and permissions using standard RBAC. Endpoints utilize a dynamic 'RoleChecker' dependency to restrict access.

```
Role    Permissions & Scope    Code Enforcement Example
```

:-

---

3. Platform Owner Portal & 2-Factor Authentication (2FA)

Organization provisioning and Super Admin creation are isolated in the Owner Portal. This administration layer is protected by multi-stage security checks:

```
'''mermaid
seque

Owner->>Portal: Input Passcode
Portal
```

2FA Verification Safeguards

- Expiration: Generated OTPs are valid for exactly 5 minutes. Any attempt to verify after this period is rejected.
- Rep

---

4. Database & Infrastructure Security

- Environment Separation: Sensitive credentials (database connection strings, JWT secret keys, SMTP passwords) are loaded dynamically from environment files ('.env') and are never committed to source control.
- SQ
autom